M1 Fire and Smoke Detection

Dataset preparation, YOLO model training plan, Kaggle workflow, external video verification, and future industry fine-tuning strategy.

1. What We Are Building

The current M1 goal is to build a visible fire and smoke detection model for industrial CCTV footage. The first model will be a base detector trained on public D-Fire data. This model is not the final plant-ready version; it is the foundation that will later be fine-tuned using actual industry camera videos and hard-negative scenes.

- Detection classes: smoke and fire.
- Detector output: bounding boxes, class labels, and confidence scores.
- Target use case: elevated industrial cameras monitoring plant areas.
- Final system direction: real-time multi-camera inference with temporal confirmation to reduce false alarms.

2. Dataset Work

For the first training stage, only the D-Fire dataset is being used. The raw dataset was kept untouched, and a clean processed YOLO dataset was generated from it. The official split files were used so that training, validation, and testing remain separated.

Split	Images	Labels	Purpose
Train	13,776	13,776	Used to learn fire and smoke detection patterns.
Validation	3,445	3,445	Used during training to select the best model and trigger early stopping.
Test	4,306	4,306	Used after training for final benchmark metrics.

- Class mapping is standardized as 0 = smoke and 1 = fire.
- Empty label files are preserved because they represent negative images with no fire/smoke.
- A small number of raw test labels had coordinates slightly outside 0 to 1; these were fixed only in the processed dataset.
- Processed dataset path: datasets/processed/fire_smoke_yolo/data.yaml.

3. Model Plan

The base model is YOLOv8n. This is selected because the M1 requirement mentions YOLOv8-nano, and because the nano model is lightweight enough for real-time CCTV deployment. Later, YOLO11n or YOLO11s can be trained as a comparison model if time and GPU availability allow.

- Baseline model: YOLOv8n pretrained weights.
- Training image size: 960, chosen to help with small fire/smoke regions in elevated camera views.
- Batch size: auto batch on Kaggle, so memory is used efficiently.
- Metrics to track: mAP50, mAP50-95, precision, recall, confusion matrix, and class-wise performance.

4. Training Workflow

Training will be performed on Kaggle. The GitHub repository will contain code only, while the processed dataset will be uploaded separately as a Kaggle input. This keeps the repository lightweight and avoids pushing large images or labels to GitHub.

Step	Action
1	Clone the GitHub repository in the Kaggle notebook.
2	Install Ultralytics and supporting Python packages.
3	Attach the processed D-Fire YOLO dataset as Kaggle input.
4	Run the Kaggle training script with GPU enabled.
5	Use validation performance to save best.pt and stop early if no new best model appears for 10 epochs.
6	Evaluate the trained model on the D-Fire test split.
7	Zip and download training outputs, graphs, weights, and evaluation results.

5. Early Stopping and Progress Records

The training script uses early stopping with patience set to 10. This means that if the validation score does not produce a new best model for 10 consecutive epochs, training stops automatically. This reduces wasted GPU time and helps avoid unnecessary overfitting.

- Best model: weights/best.pt.
- Last model: weights/last.pt.
- Progress graph: results.png.
- Training metrics table: results.csv.
- Confusion matrix and prediction examples are saved by Ultralytics when plots are enabled.
- The Kaggle training script creates a zip file of the run folder for easy download.

6. External Video Verification

A separate video will be used only after training. This video will not be included in the training, validation, or test dataset. It should be described as an external unseen verification video, mainly for qualitative demonstration of how the trained detector behaves on real footage.

- The D-Fire test split is used for benchmark metrics.
- The separate video is used for visual verification and presentation/demo output.
- The prediction script saves an annotated video, text labels, and confidence scores.
- This separation avoids data leakage and keeps the evaluation explanation honest.

7. Future Plan

After the public-data base model is trained, the next phase is industry-specific fine-tuning. The industry should provide representative CCTV footage from the actual camera locations. The most valuable footage is not only fire footage, but also normal non-fire footage that may cause false alarms.

- Request normal plant footage across day, night, rain, haze, and shift changes.
- Collect hard negatives: steam, dust, ash clouds, welding, sparks, glare, and hot equipment glow.
- Add any permitted controlled smoke/fire drill footage from the same camera height and distance.
- Fine-tune the base model on this industry-specific dataset.
- Add temporal confirmation using rolling-window voting and smoke motion checks.
- Later optimize for deployment using ONNX/TensorRT and multi-camera inference.

8. Final Position

The project is currently in the base-model stage. The dataset has been standardized, the training scripts are ready for Kaggle, early stopping is configured, progress artifacts will be saved, and external video verification is planned separately from benchmark testing. This gives a clean academic workflow now and a practical route toward real industrial deployment later.